# TP Redondance MariaDB

Réplication d'une base de données sur 2 serveurs web en répartition de charge

**Anatole PERCHER**

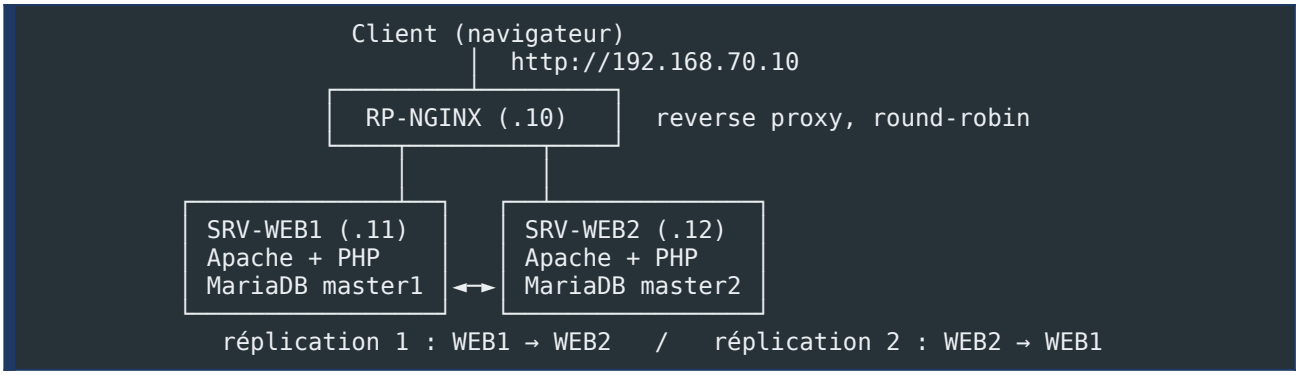
B2.2 – Installer, tester et déployer une solution d'infrastructure réseau

Domaine : **percher.ap** • 30 octobre 2026

■ **RP-NGINX** ■ **SRV-WEB1** ■ **SRV-WEB2** ■ **Réparation**

## 1. Attendus et architecture

Mettre en place une **réplication bidirectionnelle (maître-maître)** d'une base MariaDB entre deux serveurs web Apache/PHP, placés derrière un **reverse proxy Nginx** qui répartit la charge entre eux. Quel que soit le serveur qui reçoit une requête, les données écrites sont recopiées automatiquement sur l'autre.




| VM (virt-manager) | Nom d'hôte | Adresse IP | Rôle | Ressources |
|---|---|---|---|---|
| **MDB-RP-NGINX** | rp-nginx.percher.ap | 192.168.70.10/24 | Reverse proxy Nginx (répartition de charge) | 1 Go, 2 vCPU, 20 Go |
| **MDB-SRV-WEB1** | srv-web1.percher.ap | 192.168.70.11/24 | Apache + PHP + MariaDB (serveur 1) | 2 Go, 2 vCPU, 20 Go |

| VM (virt-manager) | Nom d'hôte | Adresse IP | Rôle | Ressources |
|---|---|---|---|---|
| **MDB-SRV-WEB2** | srv-web2.percher.ap | 192.168.70.12/24 | Apache + PHP + MariaDB (serveur 2) | 2 Go, 2 vCPU, 20 Go |

Les trois machines sont des Debian 12 (sans interface graphique) créées sous KVM / libvirt (virt-manager), dans le pool de stockage **TP-REDONDANCE-MARIADB** et sur le réseau virtuel **mdb-lan** (`192.168.70.0/24`, passerelle/NAT `192.168.70.1` vers Internet). Les adresses sont statiques (netplan).

## 2. Serveur Nginx (reverse proxy)

Le TP reprend l'architecture du TP Reverse Proxy : Nginx reçoit les requêtes HTTP et les transmet aux deux serveurs Apache déclarés dans un bloc `upstream`. Sans autre directive, Nginx alterne les serveurs à chaque requête (**round-robin**). Fichier `/etc/nginx/sites-available/default` :

```
# Reverse proxy + répartition de charge (round-robin) vers les 2 serveurs Apache
upstream serveurs_web {
    server 192.168.70.11;    # srv-web1
    server 192.168.70.12;    # srv-web2
}

server {
    listen 80 default_server;
    server_name rp-nginx.percher.ap;

    location / {
        proxy_pass http://serveurs_web;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
anatole@rp-nginx:~$
```

***Figure 1*** – *RP-NGINX : upstream vers srv-web1 et srv-web2, syntaxe validée par nginx -t*

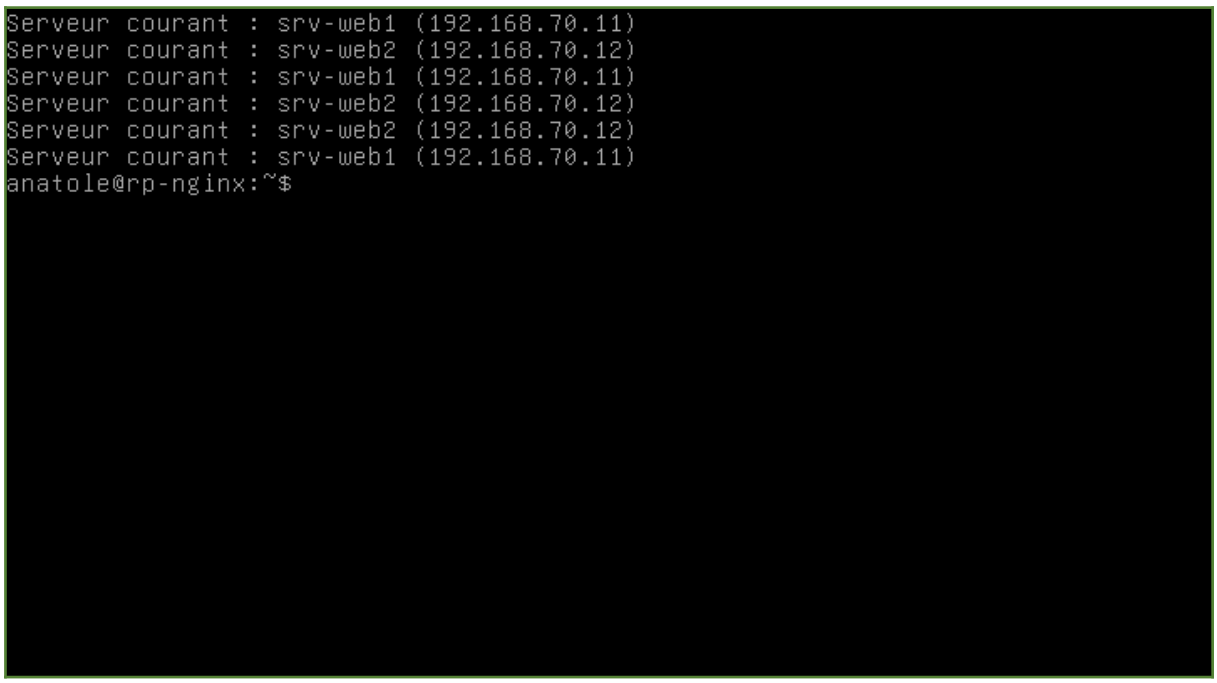



***Figure 2*** – *RP-NGINX : 6 requêtes successives, servies alternativement par srv-web1 et srv-web2*

# 3. Installation de la base de données

Sur les deux serveurs Debian :

```
sudo apt install apache2 php php-mysql
sudo apt install mariadb-server
sudo mariadb-secure-installation
```

| Question de mariadb-secure-installation | Réponse |
|---|---|
| Enter current password for root | Entrée (vide) |
| Switch to unix_socket authentication | n |
| Change the root password? | n |
| Remove anonymous users? | Y |
| Disallow root login remotely? | Y |
| Remove test database and access to it? | Y |
| Reload privilege tables now? | Y |

```
production environment.

Remove anonymous users? [Y/n] Y
 ... Success!

Normally, root should only be allowed to connect from 'localhost'.  This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] Y
 ... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access.  This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] Y
 - Dropping test database...
 ... Success!
 - Removing privileges on test database...
 ... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n]
```

*Figure 3* – *SRV-WEB1 : suppression des utilisateurs anonymes et de l'accès root distant*

```
 ... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access.  This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] Y
 - Dropping test database...
 ... Success!
 - Removing privileges on test database...
 ... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] Y
 ... Success!

Cleaning up...

All done!  If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
anatole@srv-web1:~$
```

***Figure 4*** – *SRV-WEB1 : suppression de la base test, rechargement des privilèges*

```
 ... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access.  This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] Y
 - Dropping test database...
 ... Success!
 - Removing privileges on test database...
 ... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] Y
 ... Success!

Cleaning up...

All done!  If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
anatole@srv-web2:~$
```

***Figure 5*** – *SRV-WEB2 : même sécurisation*

# 4. Mise en place de la réplication

La réplication MariaDB fonctionne en **maître → esclave** : l'esclave lit le journal binaire (binlog) du maître et rejoue les requêtes. Pour obtenir une réplication **bidirectionnelle**, on met en place deux réplications :

| | Serveur 1 (SRV-WEB1) | Serveur 2 (SRV-WEB2) |
|---|---|---|
| Réplication 1 | MASTER | SLAVE |
| Réplication 2 | SLAVE | MASTER |

## 4.1 Journal binaire et identifiant de serveur

```
sudo mkdir /var/log/mysql
sudo chown mysql:mysql /var/log/mysql
sudo chmod 750 /var/log/mysql
```

Dans `/etc/mysql/mariadb.conf.d/50-server.cnf` :

| Directive | SRV-WEB1 | SRV-WEB2 | Rôle |
|---|---|---|---|
| log_bin | /var/log/mysql/mysql-bin.log | idem | Active le journal binaire |
| bind-address | 192.168.70.11 | 192.168.70.12 | Écoute sur le réseau (et plus seulement 127.0.0.1) |
| server_id | 1 | 2 | Identifiant unique dans la réplication |
| log-basename | master1 | master2 | Préfixe des fichiers (binlog master1-bin.00000x) |
| auto_increment_increment | 2 | 2 | Ajout : pas de 2 entre les identifiants |
| auto_increment_offset | 1 | 2 | Ajout : WEB1 → id impairs, WEB2 → id pairs |

⚠️ **Remarque :** les deux directives `auto_increment_*` ne sont pas dans le sujet mais sont recommandées en maître-maître : avec la répartition de charge, deux insertions simultanées sur les deux serveurs obtiendraient le même `id` et casseraient la réplication (erreur 1062 « Duplicate entry »). Ici chaque serveur génère ses propres identifiants (impairs pour WEB1, pairs pour WEB2).

Avec `log-basename`, MariaDB nomme les binlogs `master1-bin.00000x` / `master2-bin.00000x` (dans `/var/lib/mysql`), comme dans l'exemple du sujet. Puis `sudo systemctl restart mariadb.service` sur les deux serveurs.

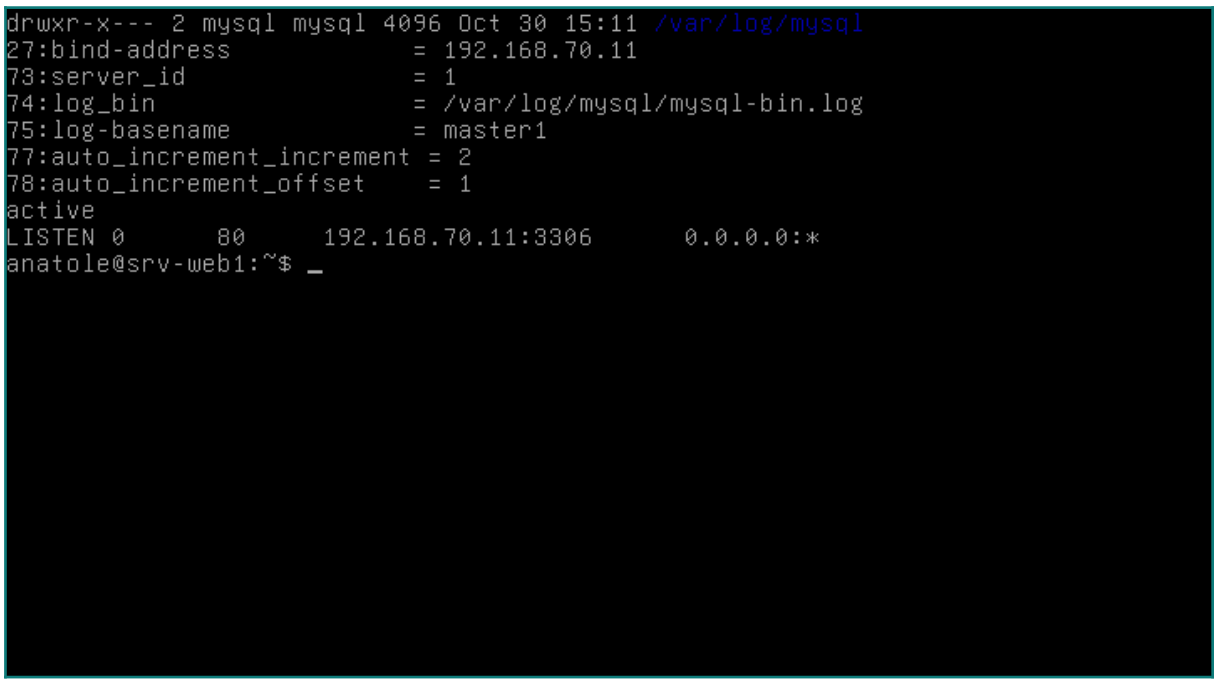



***Figure 6*** – *SRV-WEB1 : configuration de 50-server.cnf, MariaDB écoute sur 192.168.70.11:3306*

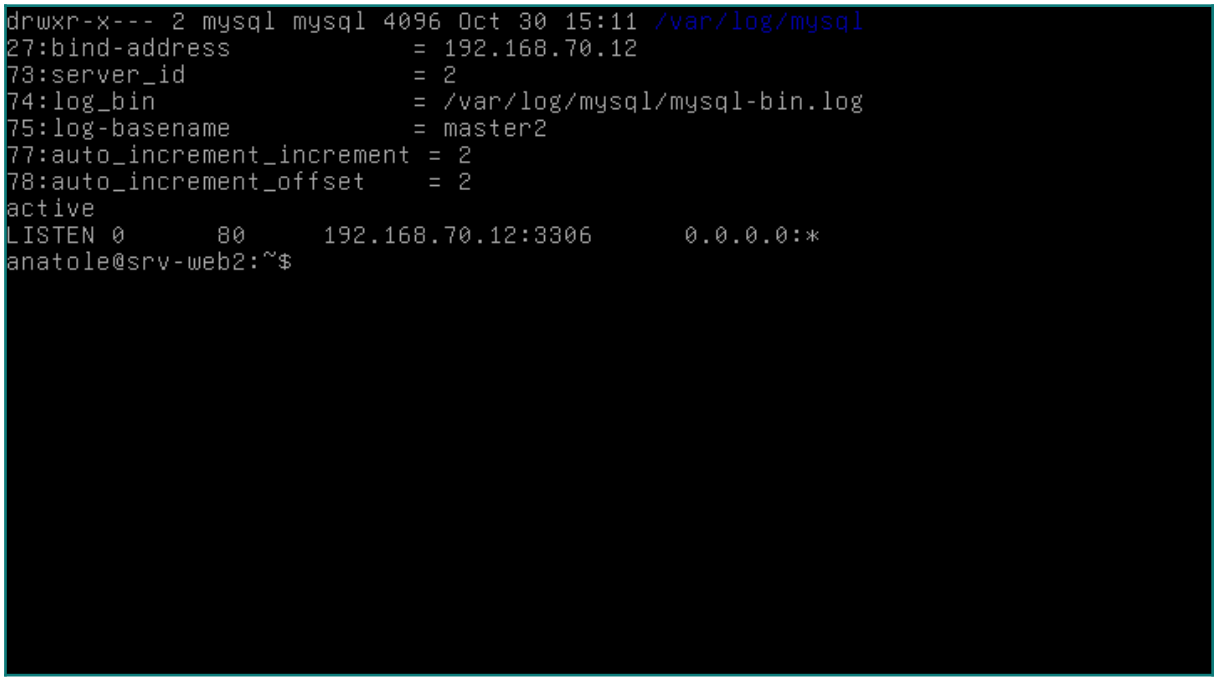



***Figure 7*** – *SRV-WEB2 : configuration de 50-server.cnf (server_id 2, master2)*

## 4.2 Réplication 1 : SRV-WEB1 MASTER → SRV-WEB2 SLAVE

Sur le **MASTER** (SRV-WEB1) : création du compte de réplication autorisé pour l'IP de l'esclave, puis relevé du fichier binlog et de la position :

```
sudo mysql -u root -p
CREATE USER 'replication_user'@'192.168.70.12' IDENTIFIED BY 'P@ssw0rd';
GRANT REPLICATION SLAVE ON *.* TO 'replication_user'@'192.168.70.12';
SHOW MASTER STATUS;
```

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 6
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE USER 'replication_user'@'192.168.70.12' IDENTIFIED BY '
P@ssw0rd'; GRANT REPLICATION SLAVE ON *.* TO 'replication_user'@'192.168.70.12';
 SHOW MASTER STATUS;
eQuery OK, 0 rows affected (0.067 sec)

Query OK, 0 rows affected (0.003 sec)

+--------------------+----------+--------------+------------------+
| File               | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+--------------------+----------+--------------+------------------+
| master1-bin.000002 |      705 |              |                  |
+--------------------+----------+--------------+------------------+
1 row in set (0.000 sec)

MariaDB [(none)]> exit
Bye
anatole@srv-web1:~$ _
```

*Figure 8* – *SRV-WEB1 (MASTER) : fichier master1-bin.000002, position 705*

Sur le **SLAVE** (SRV-WEB2), avec le fichier et la position relevés :

```
STOP SLAVE;
CHANGE MASTER TO MASTER_HOST='192.168.70.11', MASTER_USER='replication_user',
  MASTER_PASSWORD='P@ssw0rd', MASTER_LOG_FILE='master1-bin.000002',
MASTER_LOG_POS=705;
START SLAVE;
SHOW SLAVE STATUS\G
```

```
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> STOP SLAVE;
Query OK, 0 rows affected, 1 warning (0.001 sec)

MariaDB [(none)]> CHANGE MASTER TO MASTER_HOST='192.168.70.11', MASTER_USER='rep
lication_user', MASTER_PASSWORD='P@ssw0rd', MASTER_LOG_FILE='master1-bin.000002'
, MASTER_LOG_POS=705;
Query OK, 0 rows affected (0.050 sec)

MariaDB [(none)]> START SLAVE;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> exit
Bye
anatole@srv-web2:~$
```

***Figure 9*** – *SRV-WEB2 (SLAVE) : CHANGE MASTER TO vers SRV-WEB1*

```
                  Master_Host: 192.168.70.11
                  Master_User: replication_user
              Master_Log_File: master1-bin.000002
          Read_Master_Log_Pos: 705
       Relay_Master_Log_File: master1-bin.000002
            Slave_IO_Running: Yes
           Slave_SQL_Running: Yes
                 Last_Errno: 0
       Seconds_Behind_Master: 0
     Slave_SQL_Running_State: Slave has read all relay log; waiting for more u
pdates
anatole@srv-web2:~$
```

***Figure 10*** – *SRV-WEB2 : Slave_IO_Running = Yes et Slave_SQL_Running = Yes*

✔️ Les deux threads de réplication tournent : **IO** (récupère le binlog du maître) et **SQL** (rejoue les requêtes). `Seconds_Behind_Master = 0` : l'esclave est à jour.

## 4.3 Réplication 2 : SRV-WEB2 MASTER → SRV-WEB1 SLAVE

Mêmes étapes en inversant les rôles (d'abord le MASTER, puis le SLAVE) :

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 14
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE USER 'replication_user'@'192.168.70.11' IDENTIFIED BY '
P@ssw0rd';
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]> GRANT REPLICATION SLAVE ON *.* TO 'replication_user'@'192.168.
70.11';
Query OK, 0 rows affected (0.006 sec)

MariaDB [(none)]> SHOW MASTER STATUS;
+--------------------+----------+--------------+------------------+
| File               | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+--------------------+----------+--------------+------------------+
| master2-bin.000002 |      705 |              |                  |
+--------------------+----------+--------------+------------------+
1 row in set (0.000 sec)

MariaDB [(none)]> _
```

***Figure 11*** – *SRV-WEB2 (MASTER) : compte pour 192.168.70.11, master2-bin.000002 position 705*

```
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 9
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> STOP SLAVE;
Query OK, 0 rows affected, 1 warning (0.001 sec)

MariaDB [(none)]> CHANGE MASTER TO MASTER_HOST='192.168.70.12', MASTER_USER='rep
lication_user', MASTER_PASSWORD='P@ssw0rd', MASTER_LOG_FILE='master2-bin.000002'
, MASTER_LOG_POS=705;
Query OK, 0 rows affected, 1 warning (0.060 sec)

MariaDB [(none)]> START SLAVE;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]>
```

***Figure 12*** – *SRV-WEB1 (SLAVE) : CHANGE MASTER TO vers SRV-WEB2*

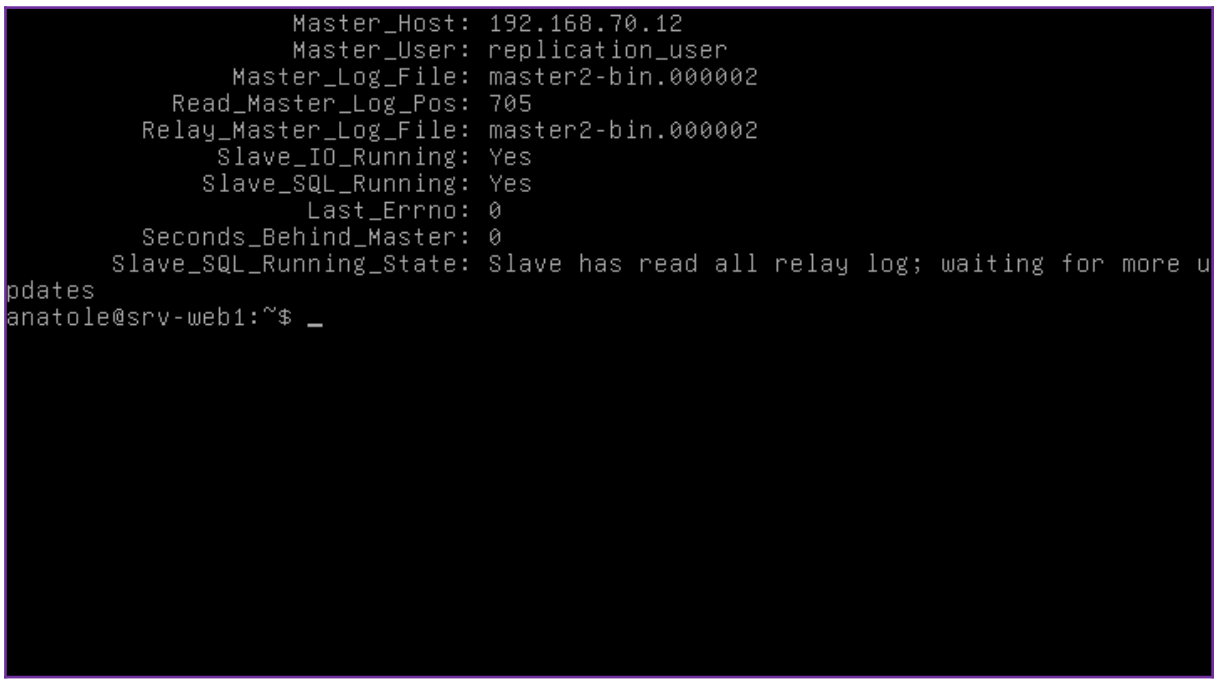



***Figure 13*** – *SRV-WEB1 : réplication 2 active (IO et SQL = Yes)*

# 5. Préparation des sites web

## 5.1 Déploiement du site sur les deux serveurs

Le contenu du zip est décompressé dans `/var/www/html` (la page par défaut `index.html` est supprimée pour que `index.php` soit servie). Le fichier `include/db_connect.php` correspond déjà à notre configuration (serveur `localhost`, utilisateur `user`, mot de passe `P@ssw0rd`, base `maBase`) : aucune adaptation nécessaire.

```
sudo rm /var/www/html/index.html
sudo unzip "BTS2_050 - Réplication BDD - TP site.zip" -d /var/www/html
sudo chown -R www-data: /var/www/html
```

```
css
dump
img
include
index.php
js
php

/var/www/html/css:
default.css

/var/www/html/dump:
conversation.sql

/var/www/html/img:

/var/www/html/include:
db_connect.php

        $servername = "localhost";
        $username = "user";
        $password = "P@ssw0rd";
        $database = "maBase";
        $conn = mysqli_connect($servername, $username, $password, $database);
anatole@srv-web1:~$
```

***Figure 14*** – *SRV-WEB1 : arborescence du site et paramètres de db_connect.php*

```
css
dump
img
include
index.php
js
php

/var/www/html/css:
default.css

/var/www/html/dump:
conversation.sql

/var/www/html/img:

/var/www/html/include:
db_connect.php

        $servername = "localhost";
        $username = "user";
        $password = "P@ssw0rd";
        $database = "maBase";
        $conn = mysqli_connect($servername, $username, $password, $database);
anatole@srv-web2:~$
```

*Figure 15* – *SRV-WEB2 : même site déployé*

## 5.2 Création de la base sur le serveur 1 uniquement

```
CREATE DATABASE maBase;
CREATE USER 'user'@'localhost' IDENTIFIED BY 'P@ssw0rd';
GRANT ALL PRIVILEGES ON maBase.* TO 'user'@'localhost';
FLUSH PRIVILEGES;

sudo mysql maBase < /var/www/html/dump/conversation.sql
```

```
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 14
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE maBase;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> CREATE USER 'user'@'localhost' IDENTIFIED BY 'P@ssw0rd';
Query OK, 0 rows affected (0.008 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON maBase.* TO 'user'@'localhost';
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]>
```

*Figure 16* – *SRV-WEB1 : création de la base maBase et de l'utilisateur user*

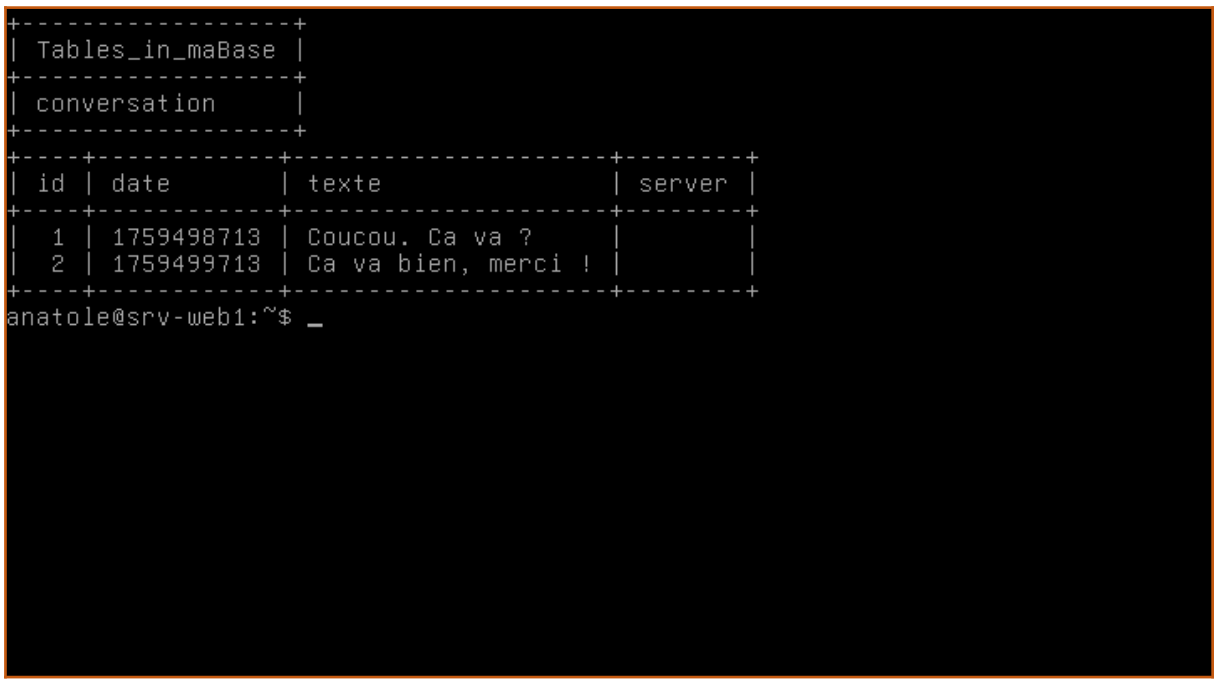



*Figure 17* – *SRV-WEB1 : import de dump/conversation.sql, table conversation créée*

## 5.3 Vérification de la réplication sur le serveur 2

✔️ **Aucune commande n'a été passée sur SRV-WEB2** : la base, la table, ses données et l'utilisateur `user` y ont été recopiés automatiquement par la réplication 1.




*Figure 18* – *SRV-WEB2 : utilisateur user, base maBase et table conversation répliqués*

## 5.4 Tests depuis le navigateur

La page affiche le serveur qui a traité la requête (« Serveur courant ») et, pour chaque message, le serveur sur lequel il a été saisi (colonne Serveur).

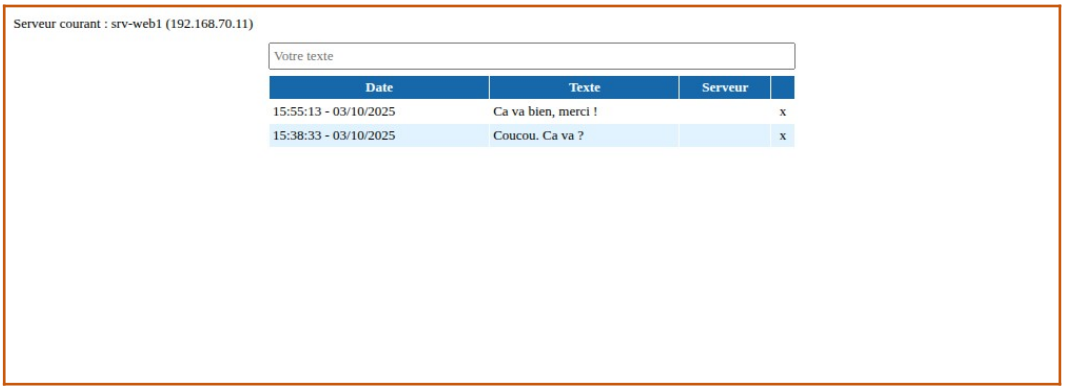



*Figure 19 – http://192.168.70.11 : le site fonctionne sur SRV-WEB1*

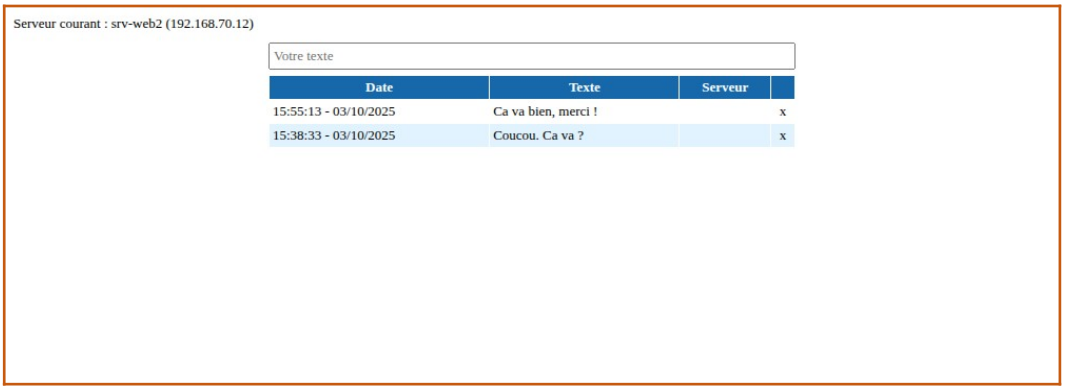



*Figure 20 – http://192.168.70.12 : SRV-WEB2 affiche les mêmes données (base répliquée)*

Un message saisi sur **http://192.168.70.11** apparaît immédiatement sur **http://192.168.70.12**, et inversement (réplication dans les deux sens) :

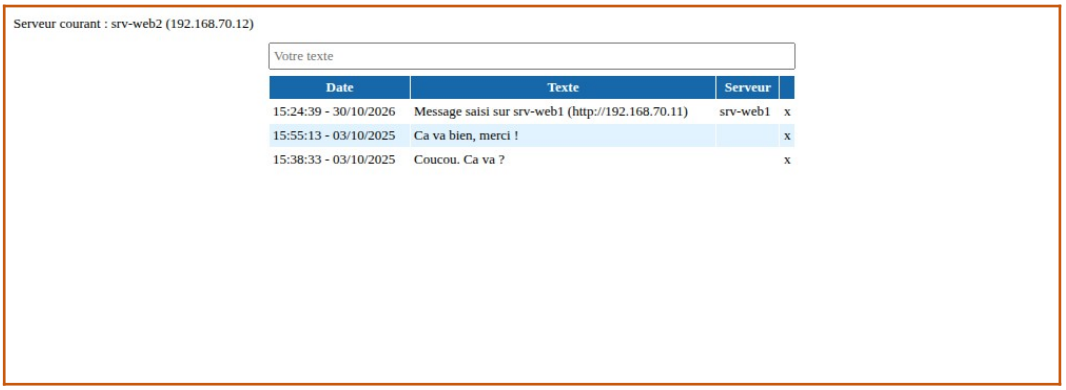



*Figure 21 – Message saisi sur srv-web1, affiché par srv-web2 (réplication 1)*

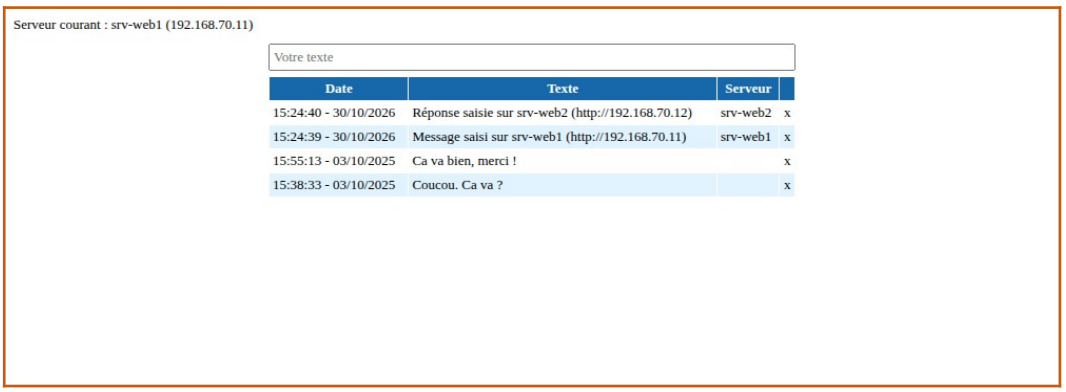



***Figure 22*** – *Réponse saisie sur srv-web2, affichée par srv-web1 (réplication 2)*

## 5.5 Test depuis le reverse proxy

Quatre messages envoyés via **http://192.168.70.10** (Nginx) : ils sont enregistrés alternativement par srv-web2 et srv-web1 (répartition de charge), et tous sont visibles quel que soit le serveur qui répond. On voit aussi l'effet de `auto_increment_offset` : identifiants impairs pour srv-web1, pairs pour srv-web2.

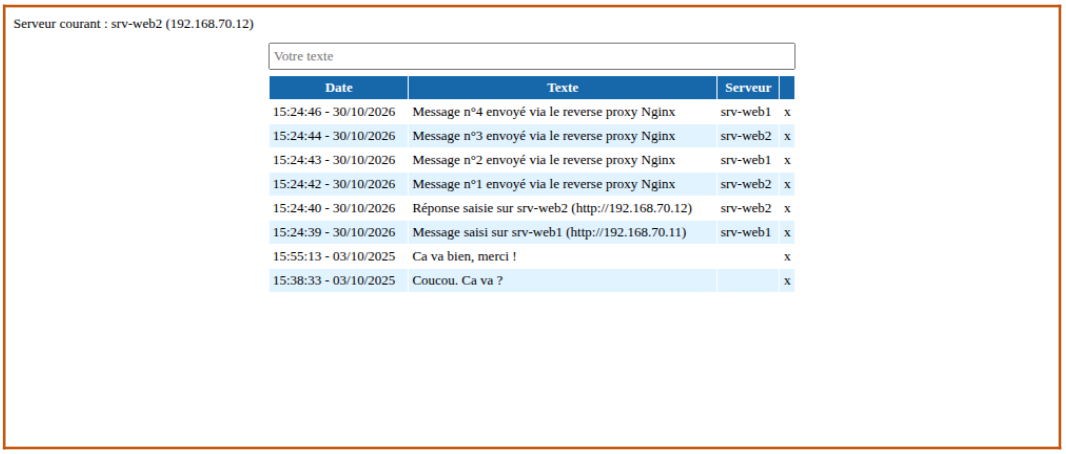



***Figure 23*** – *http://192.168.70.10 : messages répartis entre les deux serveurs, données identiques*

# 6. Réparation

i Si un décalage se crée entre les deux tables, la réplication s'arrête. Pour le démontrer, une ligne (`id` 101) a été insérée sur SRV-WEB2 **sans être journalisée** (`SET sql_log_bin=0`), puis une ligne avec le même `id` sur SRV-WEB1. Quand SRV-WEB2 rejoue cette insertion, il échoue :

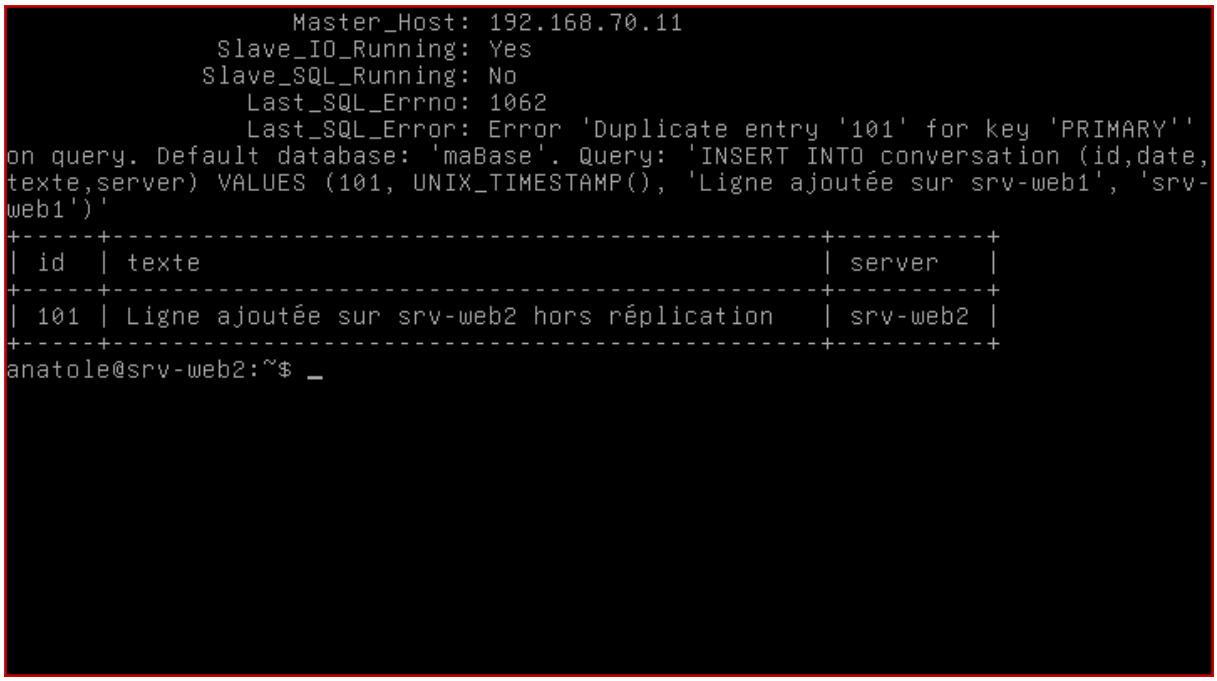



***Figure 24*** – *SRV-WEB2 : Slave_SQL_Running = No, erreur 1062 Duplicate entry '101'*

## 6.1 Export depuis le serveur où tout est correct

SRV-WEB1 contient les bonnes données : la base est exportée au format SQL.

```
sudo mysqldump maBase > ~/maBase.sql
```

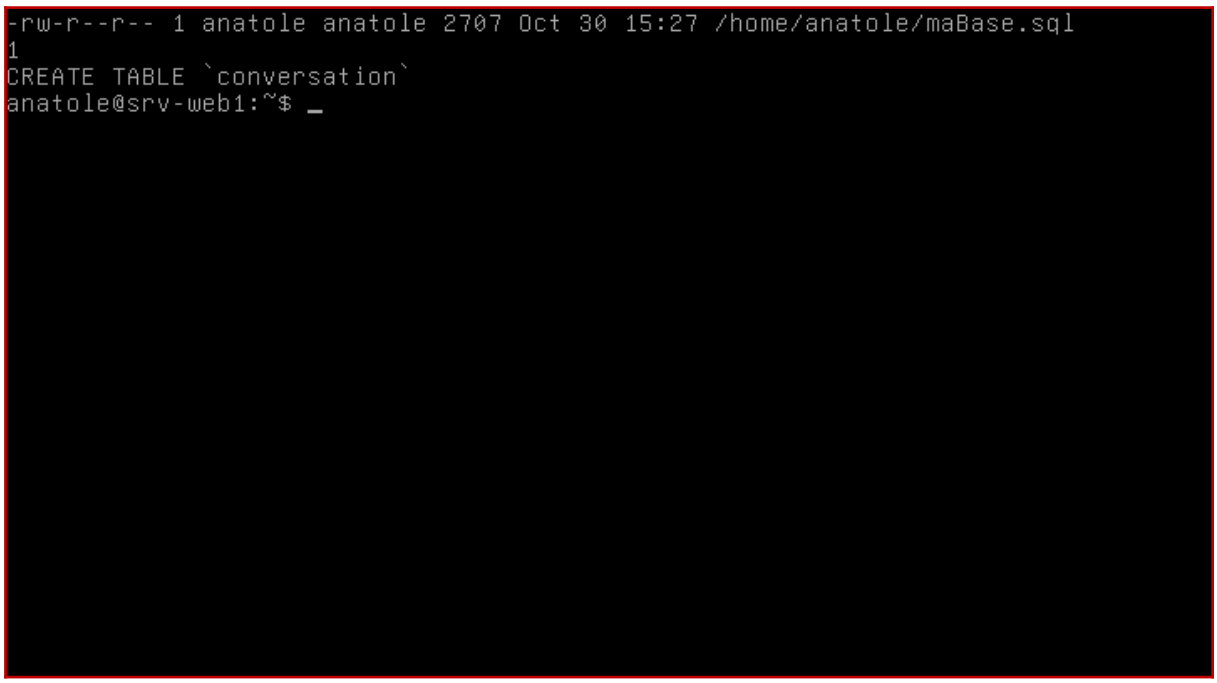



***Figure 25*** – *SRV-WEB1 : export de maBase dans ~/maBase.sql*

## 6.2 Suppression des bases et RESET MASTER

> i Comme la réplication est bidirectionnelle, la procédure est appliquée **sur les deux serveurs** (chacun est à la fois MASTER et SLAVE). Les esclaves sont arrêtés d'abord pour que les DROP ne soient pas rejoués par l'autre serveur.

```
STOP SLAVE;
DROP DATABASE maBase;
RESET MASTER;
SHOW MASTER STATUS;
```

```
Your MariaDB connection id is 52
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> STOP SLAVE;
Query OK, 0 rows affected (0.008 sec)

MariaDB [(none)]> DROP DATABASE maBase;
Query OK, 1 row affected (0.024 sec)

MariaDB [(none)]> RESET MASTER;
Query OK, 0 rows affected (0.034 sec)

MariaDB [(none)]> SHOW MASTER STATUS;
+--------------------+----------+--------------+------------------+
| File               | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+--------------------+----------+--------------+------------------+
| master1-bin.000001 |      330 |              |                  |
+--------------------+----------+--------------+------------------+
1 row in set (0.000 sec)

MariaDB [(none)]> _
```

***Figure 26*** – *SRV-WEB1 : nouveau binlog master1-bin.000001, position 330*

```
Your MariaDB connection id is 55
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> STOP SLAVE;
Query OK, 0 rows affected, 1 warning (0.001 sec)

MariaDB [(none)]> DROP DATABASE maBase;
Query OK, 1 row affected (0.038 sec)

MariaDB [(none)]> RESET MASTER;
Query OK, 0 rows affected (0.033 sec)

MariaDB [(none)]> SHOW MASTER STATUS;
+--------------------+----------+--------------+------------------+
| File               | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+--------------------+----------+--------------+------------------+
| master2-bin.000001 |      330 |              |                  |
+--------------------+----------+--------------+------------------+
1 row in set (0.000 sec)

MariaDB [(none)]> _
```

***Figure 27*** – *SRV-WEB2 : nouveau binlog master2-bin.000001, position 330*

## 6.3 RESET SLAVE et nouvelles positions

Côté SLAVE, les nouvelles valeurs relevées sur le MASTER sont utilisées (`RESET SLAVE` conserve l'hôte, l'utilisateur et le mot de passe de réplication) :

```
STOP SLAVE;
RESET SLAVE;
CHANGE MASTER TO MASTER_LOG_FILE='master1-bin.000001', MASTER_LOG_POS=330;   -- sur
SRV-WEB2
CHANGE MASTER TO MASTER_LOG_FILE='master2-bin.000001', MASTER_LOG_POS=330;   -- sur
SRV-WEB1
START SLAVE;
SHOW SLAVE STATUS\G
```

```
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 57
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> STOP SLAVE;
Query OK, 0 rows affected, 1 warning (0.000 sec)

MariaDB [(none)]> RESET SLAVE;
Query OK, 0 rows affected, 1 warning (0.048 sec)

MariaDB [(none)]> CHANGE MASTER TO MASTER_LOG_FILE='master1-bin.000001', MASTER_
LOG_POS=330;
Query OK, 0 rows affected, 1 warning (0.052 sec)

MariaDB [(none)]> START SLAVE;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> _
```

***Figure 28*** – *SRV-WEB2 : esclave repositionné sur master1-bin.000001 / 330*

```
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 55
Server version: 10.11.19-MariaDB-0+deb12u1-log Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> STOP SLAVE;
Query OK, 0 rows affected, 1 warning (0.001 sec)

MariaDB [(none)]> RESET SLAVE;
Query OK, 0 rows affected, 1 warning (0.001 sec)

MariaDB [(none)]> CHANGE MASTER TO MASTER_LOG_FILE='master2-bin.000001', MASTER_
LOG_POS=330;
SQuery OK, 0 rows affected, 1 warning (0.116 sec)

MariaDB [(none)]> START SLAVE;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> _
```

***Figure 29*** – *SRV-WEB1 : esclave repositionné sur master2-bin.000001 / 330*

## 6.4 Recréation de la base côté MASTER

```
sudo mysql -e "CREATE DATABASE maBase"
sudo mysql maBase < ~/maBase.sql
```

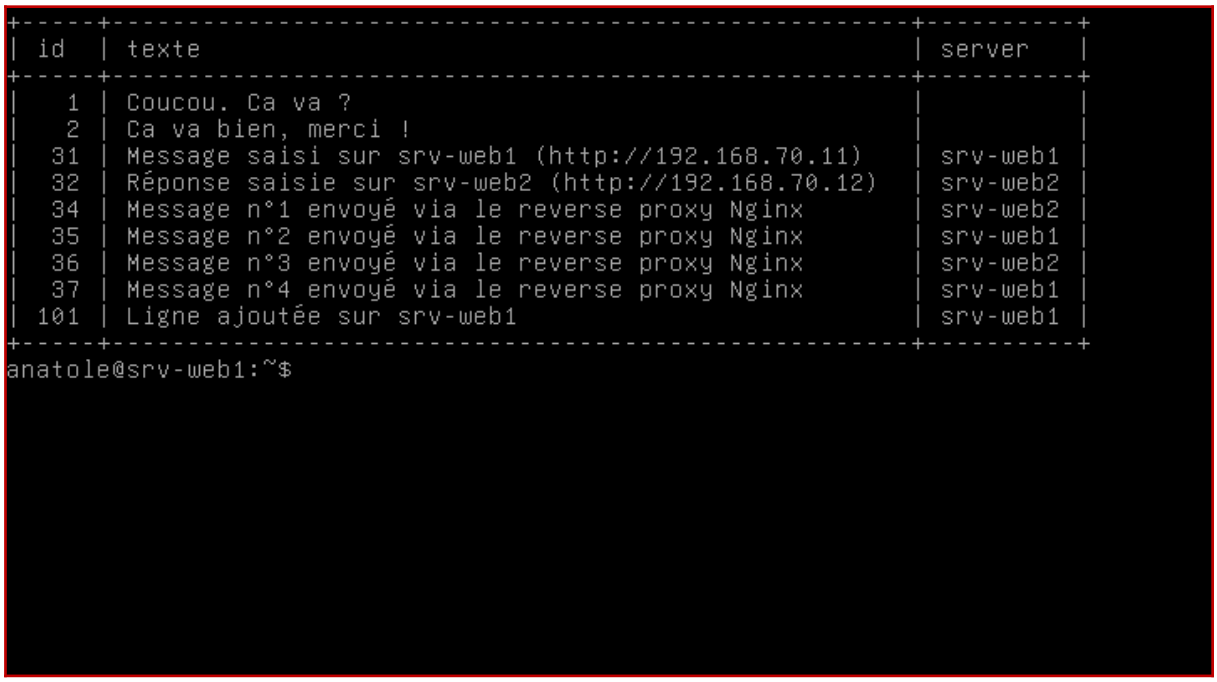



***Figure 30*** – *SRV-WEB1 : base recréée et réimportée*

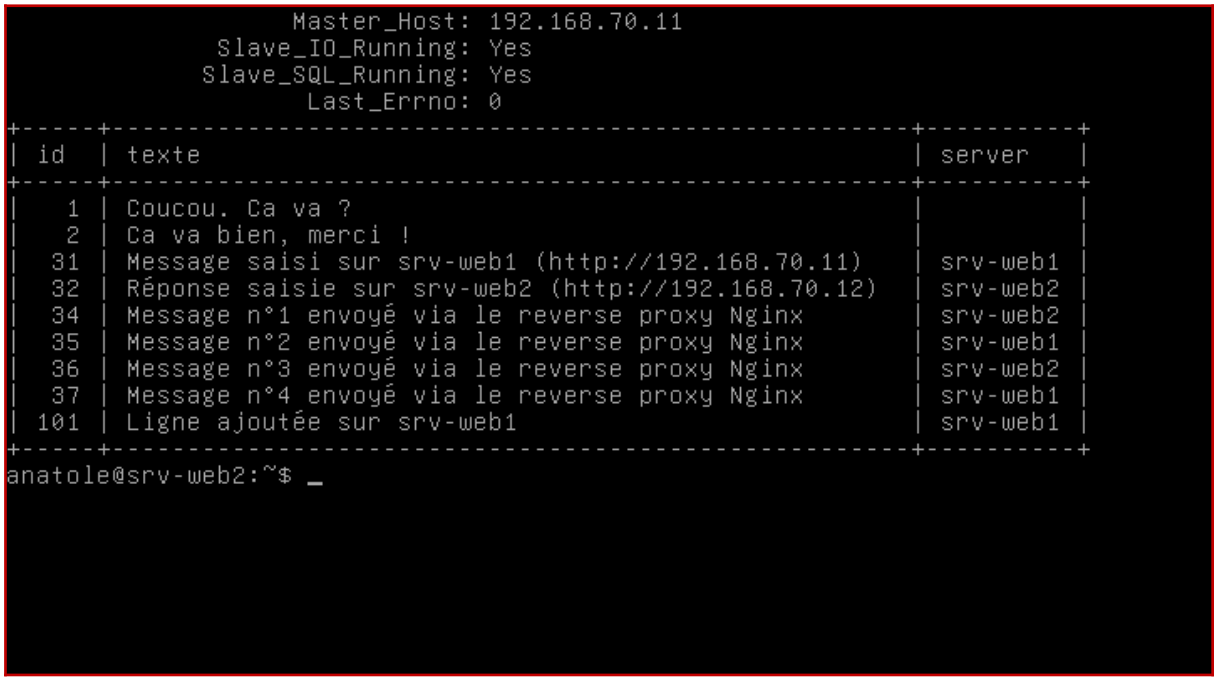



***Figure 31*** – *SRV-WEB2 : réplication relancée (IO/SQL = Yes) et données identiques, ligne 101 corrigée*

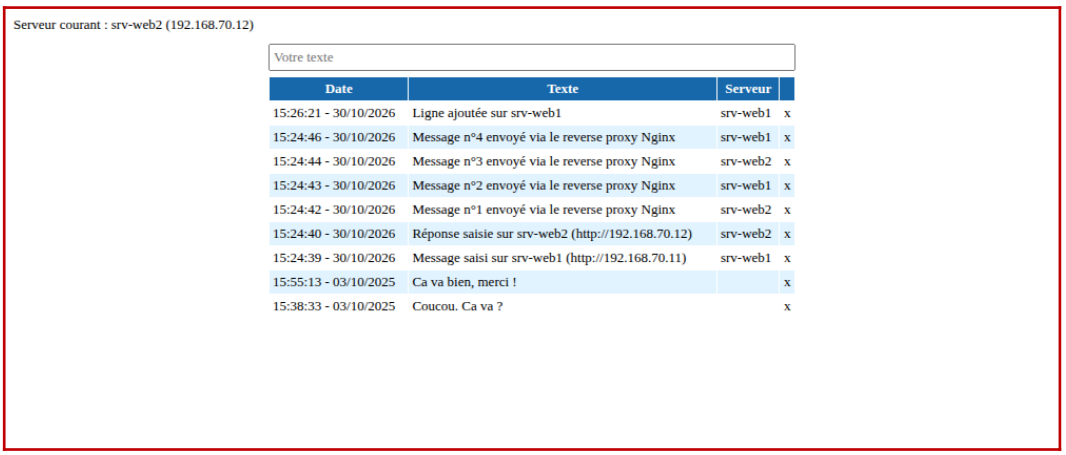



***Figure 32*** – *http://192.168.70.10 après réparation : le site fonctionne de nouveau*

✔️ La réplication fonctionne de nouveau dans les deux sens. Un redémarrage des trois VM a aussi été testé : les réplications reprennent automatiquement.

## 7. Conclusion

Les deux serveurs web disposent chacun d'une copie complète de la base `maBase`, synchronisée en temps réel par deux réplications maître → esclave croisées. Le reverse proxy Nginx répartit les visiteurs entre les deux : quel que soit le serveur qui reçoit une écriture, elle est visible partout. En cas de panne d'un serveur, l'autre dispose de toutes les données. Le point faible de cette solution est la cohérence : un écart entre les deux bases arrête la réplication et impose la procédure de réparation ; les réglages `auto_increment_*` limitent ce risque en évitant les conflits d'identifiants.